Experiment 7

Data Processing and Analysis Using Apache Pig

Subject: Data Science Lab | Environment: Hadoop 3.4.1 + Pig 0.18.0 | Mapped CO(s): CO2

1. Aim

To perform data processing and analysis on a dataset using Apache Pig and Pig Latin scripts.

2. Key Concepts

Apache Pig

High-level platform on Hadoop for analysing large datasets. Scripts are written in Pig Latin and compiled into MapReduce jobs.

Pig Latin

Data-flow language. Each statement takes a relation and produces a new relation (LOAD -> FILTER -> GROUP -> STORE).

Relation / Bag / Tuple / Field

A relation is a bag (collection) of tuples. A tuple is an ordered set of fields, like a row. A field is one value.

Grunt shell

Pig's interactive shell. Scripts can also be run in batch mode: pig -x mapreduce script.pig.

Execution modes

Local mode (-x local) reads the local file system in one JVM. MapReduce mode (-x mapreduce) reads HDFS and runs Hadoop jobs.

Lazy evaluation

Pig only builds a plan until DUMP or STORE is reached. Then the whole plan is optimised and executed.

Piggybank / CSVExcelStorage

Piggybank is Pig's library of user-contributed functions. CSVExcelStorage parses CSV files with quoted commas and can skip the header row.

3. Pig Architecture

Component	Role
Parser	Checks syntax, builds the logical plan (DAG)
Optimizer	Rewrites plan, e.g. pushes FILTER early
Compiler	Turns plan into a chain of MapReduce jobs
Execution engine	Submits jobs to Hadoop, reads/writes HDFS

4. Pig Latin Operators

Operator	Purpose	Example
LOAD	Read data	LOAD 'f' USING PigStorage(',')
DUMP	Print relation	DUMP t;
DESCRIBE	Show schema	DESCRIBE t;
FILTER	Keep matching rows	FILTER t BY Sex == 'female'
FOREACH	Transform rows	FOREACH t GENERATE Name, Fare
GROUP	Group by key	GROUP t BY Pclass

COUNT/SUM/AVG	Aggregate a bag	COUNT(t)
ORDER	Sort	ORDER t BY Fare DESC
LIMIT	First N rows	LIMIT t 5
STORE	Save to HDFS	STORE t INTO 'dir'

5. Algorithm

- 1. Start Hadoop and HDFS (NameNode, DataNode).
- 2. Upload Titanic-Dataset.csv to /user/lab7/titanic in HDFS.
- 3. Launch Pig in MapReduce mode: pig -x mapreduce script.pig.
- 4. LOAD the CSV with a schema (PassengerId, Survived, Pclass, Name, Sex, Age, ...).
- 5. DUMP the first records using LIMIT.
- 6. FILTER records on a condition (Survived == 1, Sex == 'female').
- 7. GROUP records by Pclass.
- 8. COUNT passengers in each group with FOREACH ... GENERATE.
- 9. ORDER the results.
- 10. STORE the output in a new HDFS directory and verify with hdfs dfs -cat.

6. Core Script

```
titanic = LOAD '/user/lab7/titanic/Titanic-Dataset.csv' USING CSVExcelStorage(...) AS (...);
survivors = FILTER titanic BY Survived == 1;
by_class = GROUP titanic BY Pclass;
stats = FOREACH by_class GENERATE group AS Pclass, COUNT(titanic) AS passengers;
sorted = ORDER stats BY passengers DESC;
STORE sorted INTO '/user/lab7/output/class_stats' USING PigStorage(',');
```

7. Results (Titanic, 891 passengers)

Pclass	Passengers	Survived	Avg Fare	Survival %
1	216	136	84.15	62.96
2	184	87	20.66	47.28
3	491	119	13.68	24.24

Survivors (FILTER Survived == 1): 342. Female passengers: 314.

8. Exercise Answers

Exercise 1: Filter only female passengers

females = FILTER titanic BY Sex == 'female'; -> 314 records.

Exercise 2: Count passengers in each class

GROUP titanic BY Pclass; FOREACH ... GENERATE group, COUNT(titanic); -> 1: 216, 2: 184, 3: 491.

Exercise 3: Sort passengers by Fare (descending)

ORDER fare_view BY Fare DESC; -> highest fare 512.33 (Ward, Cardeza, Lesurer - all 1st class); lowest fares are 0.0.

Exercise 4: Store output in a new HDFS directory

STORE female_summary INTO '/user/lab7/exercises/ex4_female_summary' USING PigStorage(','); The directory contains _SUCCESS and part-r-00000. Female survival: 1st 96.81%, 2nd 92.11%, 3rd 50.0%.

Exercise 5: FILTER vs FOREACH

	FILTER	**FOREACH ... GENERATE**
Works on	Rows (tuples)	Columns (fields)
Purpose	Select rows by condition	Transform / project each row
Row count	Same or fewer	Same as input
Columns	All kept	Chosen / computed
SQL analogy	WHERE	SELECT list
Example	FILTER t BY Age > 70	FOREACH t GENERATE Name, Fare*2

Demo: FILTER Age > 70 gave 5 rows x 12 columns; FOREACH on that gave 5 rows x 4 new columns.

9. Viva Q&A

Q: What is Apache Pig?

A: A high-level data-flow platform on Hadoop that runs Pig Latin scripts as MapReduce jobs.

Q: Why use Pig instead of Java MapReduce?

A: Far fewer lines of code, built-in operators (JOIN, GROUP, ORDER), and automatic optimisation.

Q: What is a relation in Pig?

A: A bag of tuples, similar to a table, with an optional schema.

Q: Difference between a bag and a tuple?

A: A tuple is one ordered record; a bag is an unordered collection of tuples.

Q: Local vs MapReduce mode?

A: Local uses the local file system in one JVM; MapReduce uses HDFS and Hadoop jobs.

Q: What does DUMP do?

A: Executes the plan and prints a relation to the console.

Q: DUMP vs STORE?

A: DUMP prints to screen; STORE writes the relation into a file system directory.

Q: What does GROUP produce?

A: Tuples of (group, bag of matching tuples), e.g. (1, {all 1st-class rows}).

Q: Why CSVExcelStorage?

A: Names contain commas inside quotes; PigStorage(',') would split them incorrectly.

Q: What is lazy evaluation in Pig?

A: Statements only build a plan; nothing runs until DUMP or STORE.

Q: What is a UDF?

A: User Defined Function written in Java/Python to extend Pig with custom logic.

Q: Pig vs Hive?

A: Pig is a procedural data-flow language (good for ETL); Hive is declarative SQL (good for reporting).

Q: What is DESCRIBE?

A: Shows the schema of a relation.

Q: Can STORE overwrite an existing directory?

A: No. The output directory must not exist, or the job fails.

10. One-Line Definitions

Pig Latin

Data-flow scripting language of Apache Pig

Relation

Bag of tuples (Pig's table)

Tuple

Ordered set of fields (a row)

Bag

Collection of tuples

Grunt

Interactive Pig shell

LOAD

Reads data into a relation

FILTER

Selects rows by condition

FOREACH

Processes each row and generates fields

GROUP

Collects rows with the same key

STORE

Writes a relation to HDFS